

AI Internship Sprint

Day 1 Study Guide — Backend, APIs & Generative AI

This guide covers everything built and learned on Day 1: web architecture & REST APIs, the Task Tracker API (Project 1), the ML Prediction API (Project 2), and Generative AI fundamentals. Each topic is explained in plain language with a real example from the projects built today, followed by a bank of likely interview questions with model answers.

What was built today	Status
Project 1 — Task Tracker API (FastAPI + SQLite, full CRUD)	Done
Project 2 — ML Prediction API (Iris classifier + DB logging)	Done
Notes — Web Architecture	Done
Notes — Generative AI	Done

1. Web Architecture & REST APIs

What is an API?

Definition:

A defined set of rules that lets two programs talk to each other — what requests you're allowed to send, and what responses you'll get back. Think of it as a contract between a client (the requester) and a server (the responder).

Where it's used:

Virtually every modern app: mobile apps talking to backends, websites fetching data, one service calling another service.

Example:

Swagger UI sends a POST request to our Task Tracker API, and the API responds with the newly created task as JSON.

Key terms:

endpoint, client, server, request, response.

Endpoint

Definition:

A specific URL + HTTP method combination that triggers exactly one function on the server. The same URL text can be two completely different endpoints depending on the method used.

Example:

GET /tasks (list all tasks) and POST /tasks (create a task) share the same URL path but are two separate, independent routes.

Request & Response

Definition:

A request is everything the client sends: the method, the URL, optional headers, and an optional body (usually JSON). A response is everything the server sends back: a status code, headers, and usually a body.

Example:

POST /predict sends 4 flower measurements as the request; the response is JSON containing the predicted species and confidence score.

HTTP Methods

Definition:

A convention for describing the *type* of action a request performs, so both the server code and anyone reading it instantly knows the intent.

Key terms:

GET = retrieve data, don't change anything. POST = create something new. PUT = update/replace an existing resource. DELETE = remove a resource.

Status Codes

Definition:

A 3-digit number in every response telling the client what happened, without needing to read the body to find out.

Example:

GET /tasks/999 (a task that doesn't exist) correctly returns 404 with {"detail": "Task not found"}.

Key terms:

200 = success. 404 = Not Found (the request was understood, but the thing asked for doesn't exist). 422 = Unprocessable Entity (the request body itself failed validation — wrong types, missing required fields).

JSON

Definition:

A lightweight, text-based format for structuring data as key-value pairs. It's the standard format almost all modern APIs use to send and receive data because it's easy for both humans and machines to read.

Example:

{"title": "Finish project", "status": "pending", "priority": "high"}

REST

Definition:

A design convention for APIs built around resources (nouns, like "tasks" or "predictions") combined with standard HTTP methods (verbs) acting on them — instead of inventing a custom, one-off endpoint name for every single action.

Example:

/tasks is the resource; GET/POST/PUT/DELETE on it cover every CRUD action, rather than endpoints like /createTask, /getAllTasks, /removeTaskById, etc.

CRUD

Definition:

Create, Read, Update, Delete — the four basic operations almost every data-backed application needs to support.

Why it matters for this internship:

This is exactly what Project 1 (Task Tracker API) implements end-to-end.

Long Polling vs. WebSockets

Definition:

Two different strategies for getting near-real-time updates from a server.

Where it's used:

Long polling: simpler systems, legacy infrastructure, infrequent updates. WebSockets: chat apps, live dashboards, multiplayer games, collaborative editing tools.

Key terms:

Long polling: the client repeatedly asks "anything new?" — the server holds the request open until there's new data (or a timeout), responds, and the client immediately asks again. Simple to implement, but has repeated request/response overhead. WebSockets: a single persistent, two-way connection — either side can send data anytime without re-establishing a connection. More efficient for frequent, real-time updates.

Why it matters for this internship:

Not built this sprint, but a very common interview topic when discussing API design tradeoffs and real-time systems.

2. FastAPI + Database — Task Tracker API (Project 1)

FastAPI & Uvicorn

Definition:

FastAPI is a Python web framework — it defines routes and what happens when each is hit. Uvicorn is the actual server: it opens a network port, listens for incoming requests, and hands each one to FastAPI to handle, then sends the response back over the network.

Example:

Analogy: FastAPI is the recipe book and chef; Uvicorn is the building, front door, and waiter actually serving customers. You need both.

Swagger UI (/docs)

Definition:

FastAPI automatically reads every registered route and generates an interactive webpage from it, letting you test endpoints directly in the browser without writing a separate test client.

Why it matters for this internship:

Used throughout Day 1 to test every CRUD endpoint and inspect real request/response bodies — a completely legitimate way to answer "how did you test your API?"

SQLModel & the ORM Pattern

Definition:

An ORM (Object-Relational Mapper) lets you define a database table as a Python class, and work with rows as Python objects instead of writing raw SQL. SQLModel combines this with Pydantic for automatic validation.

Example:

class Task(SQLModel, table=True): id, title, description, status, priority, created_at — this class is literally the tasks table.

Pydantic Validation & Typed Parameters

Definition:

When a function parameter is typed as a model (e.g. `task: Task`), FastAPI automatically parses the incoming JSON, validates every field's type against the model, and either hands back a clean, ready-to-use object or rejects the request with a 422 — all before your function code runs.

Example:

Sending a number for "priority" (which expects a string) is rejected automatically with a 422, with zero manual validation code written.

The Session Pattern (add → commit → refresh)

Definition:

The standard sequence for writing to a database safely.

Example:

Creating a task: add it, commit it to `tasks.db`, then refresh so the response can include the real database-assigned id.

Key terms:

`session.add(obj)` stages the change in memory (not saved yet). `session.commit()` actually writes it to the database file. `session.refresh(obj)` pulls back any database-generated values (like the auto-assigned id) into your Python object.

Path Parameters & 404 Handling

Definition:

A path parameter (like `{task_id}` in `/tasks/{task_id}`) captures part of the URL and passes it into the function, auto-converted to the declared type. Manual 404 handling means explicitly checking if the requested row exists before returning it.

Example:

if not task: raise `HTTPException(status_code=404, detail="Task not found")` — returns a clean, correct error instead of crashing or returning an empty body silently.

3. Machine Learning — ML Prediction API (Project 2)

Train/Test Split

Definition:

Splitting data into a training set (the model learns from this) and a test set (used only to check accuracy, never seen during training). This prevents a model from looking accurate just because it memorized the data it was trained on.

Example:

`train_test_split(X, y, test_size=0.2)` keeps 20% of the Iris dataset hidden from training, purely to evaluate honestly afterward.

Logistic Regression (Classification)

Definition:

Despite the name containing "regression," this is a classification algorithm — it predicts which category (class) something belongs to, based on input features.

Example:

Given 4 flower measurements, the model predicts one of 3 species: setosa, versicolor, or virginica.

predict() vs. predict_proba()

Definition:

predict() returns only the final answer — the single most likely class. predict_proba() returns a probability for every possible class, letting you report a confidence score instead of just a flat answer.

Example:

predict_proba() might return [0.02, 0.95, 0.03] — predict() would just return the class with the highest of those three numbers.

Model Serialization (joblib)

Definition:

Saving a trained model's learned internal state (its weights and biases) to a file, so it can be loaded instantly later without retraining. The file is binary, not human-readable — it's essentially the model's learned coefficients and intercepts, frozen in place.

Why it matters for this internship:

iris_model.joblib is loaded once at server startup (not inside the /predict function) so predictions are fast and the model isn't rebuilt on every request.

Overfitting

Definition:

When a model memorizes specifics of its training data rather than learning a generalizable pattern — it performs great on data it has seen, but poorly on new, unseen data.

Example:

Our Iris model scored 1.00 test accuracy — not a red flag here because Iris is a small, easily separable dataset, but in a messier real-world dataset, 100% accuracy would usually be suspicious.

Key terms:

Classic warning sign: a large gap between training accuracy and test accuracy (e.g. 99% train vs. 60% test).

Cross-validation (testing across multiple different data splits) gives a more reliable accuracy estimate than a single split.

4. Generative AI

What is Generative AI?

Definition:

AI systems that create new content (text, images, code, audio, video) rather than just classifying or predicting a fixed label. Trained on large amounts of existing data to learn patterns, then generate new outputs that follow those patterns.

Where it's used:

Chatbots and writing assistants, code generation/autocomplete, image generation, voice synthesis, video generation.

Key Terms

Definition:

The vocabulary used when discussing how generative models work.

Key terms:

Prompt = the input/instruction given to the model. Token = a chunk of text (roughly a word or part of one) the model processes one at a time. Hallucination = confidently generating false or made-up information. Fine-tuning = further training a pre-trained model on specific data to specialize it. Temperature = a setting controlling how random/creative vs. predictable the output is.

Risks

Definition:

The main practical concerns when using generative AI in real applications.

Key terms:

Hallucination — plausible-sounding but false output (invented facts, citations). Bias — models can reproduce biases present in their training data. Overreliance — treating generated output as automatically correct without verification. Misuse — generating misinformation or deceptive content at scale.

Prompting Basics

Definition:

Practical techniques for getting better output from a generative model.

Key terms:

Be specific and provide context — vague prompts get vague answers. Give examples of the desired output format (few-shot prompting). Break complex tasks into smaller steps. Iterate — refine the prompt based on what the previous output got wrong.

5. Likely Interview Questions & Answers

Practice saying these out loud, in your own words, before the interview. The goal is fluency, not memorization.

Q: Walk me through what happens when a client sends POST /tasks.

A: The client sends a POST request to /tasks with a JSON body. Uvicorn receives it and hands it to FastAPI, which matches it to the create_task function. Before that function runs, FastAPI validates the JSON against the Task model (Pydantic) — if validation fails, it returns a 422 immediately. If it passes, the function opens a database session, adds the new task, commits it (writing to disk), refreshes to get the auto-assigned id, and returns the task as JSON back to the client.

Q: What's the difference between a 404 and a 422 error?

A: 404 means the request was correctly formed, but the specific thing requested doesn't exist (e.g. GET /tasks/999 with no task 999). 422 means the request itself is malformed — wrong data types or missing required fields — and gets rejected by validation before the endpoint's own logic even runs.

Q: Why type function parameters as a model (e.g. task: Task) instead of parsing raw JSON?

A: It offloads validation work to FastAPI/Pydantic automatically — type checking, required fields, and clean error responses (422) all happen for free. It also makes the API self-documenting, since /docs reads the expected request shape directly from the model.

Q: If you restarted your server, would the data still be there? Why?

A: Yes — the data lives in a SQLite file on disk (tasks.db), not in memory. Restarting the server just reloads the Python code; it doesn't touch the database file. Data only disappears if the database file itself is deleted.

Q: Why load your ML model once at startup instead of inside the /predict function?

A: Loading from disk and rebuilding the model is unnecessary work to repeat on every single request. Loading once at startup keeps the model in memory, ready to reuse instantly for every prediction — this is the standard 'load once, serve many' pattern.

Q: What's the difference between predict() and predict_proba()?

A: predict() returns only the single most likely class. predict_proba() returns a probability for every possible class, which lets the API report a genuine confidence score instead of a flat answer.

Q: Is 100% model accuracy always a good sign?

A: Not necessarily — it can be a red flag for overfitting on a messy real-world dataset. In this case it's expected, because Iris is a small, well-separated toy dataset; on a noisier real-world dataset I'd be suspicious of perfect accuracy and would check the gap between training and test accuracy, or use cross-validation.

Q: What's the difference between classification and regression?

A: Classification predicts a category/label (e.g. which of 3 flower species). Regression predicts a continuous number (e.g. a price or temperature). Logistic Regression, despite the name, is actually a classification algorithm.

Q: What's REST, in your own words?

A: A convention for designing APIs around resources (nouns like 'tasks') combined with standard HTTP methods (verbs) acting on them, instead of inventing a separate custom endpoint name for every action.

Q: When would you use WebSockets instead of long polling?

A: WebSockets when you need frequent, low-latency, two-way updates — like a chat app or live dashboard — because the connection stays open and avoids repeated request overhead. Long polling is simpler to implement and fine for less frequent or less latency-sensitive updates.

Q: What is hallucination in the context of generative AI?

A: When a model confidently generates output that sounds plausible but is actually false or fabricated — for example, inventing a citation or a fact that doesn't exist, while stating it as if it were true.

Q: How did you test your API endpoints without writing a separate test client?

A: FastAPI auto-generates an interactive Swagger UI at /docs from the registered routes, which lets you send real requests and see real responses directly in the browser — that's what I used to test every CRUD operation and the prediction endpoint.